

Project Proposal

1. Predict Potential Spammer

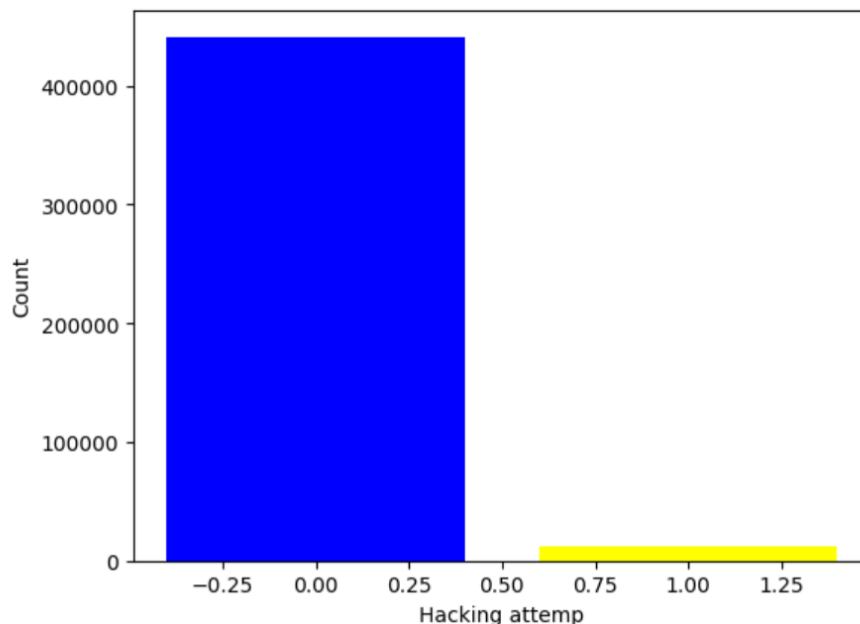
2. **Brief on the project:** This project detects probability of a registered user being a spammer. This means you are given historical data about users — including certain features about how they behave when signing up and shortly thereafter — and you must train a model to identify which users are likely to engage in spammy behavior later.

3. **Deliverables of the project:** As part to solve this project we will be performing following activities.

- Import data into python env using panda.
 - Perform basic EDA activities like removing duplicates, treating null values, treating outliers and so on.
 - Test data against different models. Capture accuracy, precision, recall, f1-score on training and testing data.
- List of questions your model/problem are designed to answer.
This model typically answers one question, whether the fiverr user is potential spammer or not. For this model is trained on approx 4.5 lacs or records.
 - Details of the model , important findings, expecting observations and outcome.
Evaluate different models and finalize the best model based on accuracy, precision and recall values.

We evaluated several models and found RandomForest to be the best in terms of accuracy, precision and recall values for Minority class. Also had several observations:

- User_id is irrelevant column and we removed it.
- There were duplicate records which we removed.
- We observed minority class records are just 2.7 % of entire records. There is huge class imbalance. We used SMOTE method to treat class imbalance.



- There were few null values in X13 column and we treated them by mode.
- We treated outliers by applying filter at 1 and 99 quantile. We capped values of smaller than 1 quantile and larger than 99 quantile at 1 and 99 quantile respectively.
- After outlier treatment we noticed several duplicate columns and we removed them.
- We observed skewness and treated it with PowerTransformer
- We observed different scales for feature variables, therefore performed standard scaling
- We observed too many features, and tried PCA. 2 PCA components very well explained the variance, but PCA components performed poorly on accuracy, recall and precision of minority class.

4. Resources

- **Data set source:** We have used dataset provided at Kaggle website.
<https://www.kaggle.com/competitions/predict-potential-spammers-on-fiverr/>
- **Software:** We have used variety of softwares and packages for this project:
 - a) Python
 - b) Panda
 - c) Sci-kit learn
 - d) Numpy
 - e) Visual studio code
 - f) Matplotlib
 - g) Seaborn

5. Individual Details:

Name: Wasim Bari

Mobile: 9650307130

Email: wbari31@gmail.com

6. Milestones

Milestones
Define a problem
Understanding the business problem
Get the Data
Explore and pre-process data
Choosing the python platform
Create Features
EDA

Milestones
Create Model
Model Evaluation
Report Writing
Project submission

Define a problem:

A user registers on fiverr platform. A user can be genuine user or fraudster spammer. It identifies whether a user is spammer or not based on the behavior during registration or soon after.

Understanding the business problem:

While registration user provides several details. Based on the values filled in by the user during registration, user's behavior pattern is identified. Different fields captured are provided to the data scientist for developing an ML model to predict whether the user is spammer or not. For the privacy and safety of the user, actual values are encoded. Also column names are masked so that outside entity cannot extract meaningful personal information from the data provided.

Getting data:

Data is available at Kaggle website.

EDA:

As part of EDA we performed following activities:

- Loading data into panda.

```
df=pd.read_csv('fiverr_data.csv')  
df.shape
```
- Dropping irrelevant column 'user_id'

```
df=df.drop('user_id', axis=1)
```
- Dropping duplicate records from dataset.

```
# Drop duplicates  
df=df.drop_duplicates()
```
- Finding null values.

```
df.isna().sum()[df.isna().sum() > 0]
```
- Outlier treatment

```
#Outlier Treatment. Not treating outlier for now.
#Treating outliers
def list_treat_outliers_quantile(df, col, lower_quantile,
upper_quantile):

    lower = df[col].quantile(lower_quantile)
    upper = df[col].quantile(upper_quantile)
    df_outliers=df[(df[col]<lower) | (df[col]>upper)]
    outlier_count=df_outliers.shape[0]
    total_count=df.shape[0]
    df[col] = df[col].clip(lower, upper)
    ratio=outlier_count/total_count

for col in df.select_dtypes(include='number').columns:
    list_treat_outliers_quantile(df, col, .01,.99)
```

- **Finding and removing duplicate columns.**

```
# Find duplicate columns.
def find_duplicate_columns(df):
    duplicates = {}
    seen = {}

    for col in df.columns:
        col_values = tuple(df[col])

        if col_values in seen:
            duplicates.setdefault(seen[col_values], set()).add(col)
        else:
            seen[col_values] = col

    return duplicates

duplicate_cols=find_duplicate_columns(df)
duplicate_cols

# Removing duplicate columns
df=df.drop(columns=['X12','X26','X27','X29','X30','X33','X37','X45','X4
6','X47','X48','X51'], axis=1)
- Separating target variable with features
# Split data into X and Y
# Changing type for X13 from float to int
df['X13']=df['X13'].astype(int)
X=df.drop('label', axis=1)
y=df['label']
```

- Doing train and test split.

```
# Train and test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25, random_state=3)
```

- Finding and treating skewness.

```
# find skewness
X.skew().sort_values(ascending=False)
# Treating skewness
skewed_cols=['X3', 'X6', 'X9', 'X13', 'X14', 'X15', 'X17', 'X19', 'X21', 'X23', '
X24', 'X25', 'X28', 'X31', 'X32',
            'X34', 'X35', 'X38', 'X39', 'X40', 'X41', 'X42', 'X43', 'X44']
from sklearn.preprocessing import PowerTransformer

# Apply PowerTransformer to skewed columns
pt = PowerTransformer(method='yeo-johnson')
X_train[skewed_cols] = pt.fit_transform(X_train[skewed_cols])
X_test[skewed_cols] = pt.transform(X_test[skewed_cols])
```

- Doing feature scaling on feature variables.

```
# Scale X_train and X_test using StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

- Performing oversampling using SMOTE method on training data. I tried SMOTETOMEK, but it gave poor results.

```
#Performing oversampling
from imblearn.combine import SMOTETomek
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy=0.9, random_state=3)
X_train_smt, y_train_smt = smote.fit_resample(X_train, y_train)
display(y_train_smt.value_counts())
```

- We tried but did not implement PCA, since it produced poor matrix.

Model Evaluation:

We used testing against following models:

- XGBoost
- Random Forest

- DecisionTree
- Logistic Regression
- KNN
- Gaussian Naïve Bayes
- SVM

Given below is the table for model comparison:

S. No	Model	Accuracy		Precision 0		Recall 0		Precision 1		Recall 1	
		Training	Testing	Training	Testing	Training	Testing	Training	Testing	Training	Testing
1	XGBoost	0.96	0.97	0.97	0.99	0.97	0.97	0.97	0.42	0.97	0.75
2	Random Forest	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.64	0.99	0.68
3	Decision Tree	0.91	0.92	0.9	0.99	0.93	0.93	0.92	0.23	0.89	0.77
4	KNN	0.98	0.94	1	0.99	0.96	0.94	0.96	0.27	1	0.75
5	Naïve Bayes	0.79	0.71	0.87	1	0.71	0.71	0.73	0.08	0.88	0.88
6	Logistic Regression	0.86	0.87	0.87	0.99	0.87	0.87	0.86	0.16	0.85	0.84
7	SVM	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

Final decision: Random Forest gives the highest Precision and Recall values for 1 categorical value.

SVM did not complete run. It was taking too long to complete.